



Curso Técnico del Sistema KAVAC

MÓDULO 4: CAPA PROXY

Lección 1: Proxy

Importancia de un Proxy:

Utilizar un proxy inverso como Nginx en la puesta en producción de una aplicación web que maneja datos sensibles es crucial para mejorar la seguridad, rendimiento y escalabilidad de la aplicación. Un proxy no solo protege los servidores backend de ataques y sobrecargas, sino que también optimiza la entrega de contenido y facilita la administración de la infraestructura. Implementar un proxy inverso correctamente configurado es una de las mejores prácticas esenciales para asegurar la integridad y disponibilidad de datos en una aplicación web.

Tomando en cuenta que el kavac es un aplicación que guarda datos confidenciales de la institución o empresa es necesario que en el despliegue se tome en cuenta un proxy inverso.

En caso de tener un servidor proxy inverso puedes adaptar la configuración del manual en la sección III (Instalar y configurar la capa Proxy) al proxy existente, de lo contrario accede a la máquina virtual que vas a utilizar como proxy y comienza la instalación guiándote por el manual y el video tutorial.

Nginx como servidor:

Nginx se utiliza como proxy inverso en la aplicación KAVAC para manejar diversas responsabilidades críticas que optimizan la seguridad, el rendimiento y la escalabilidad de la aplicación.

Descripción de cómo y por qué se usa Nginx como proxy inverso en KAVAC:

- **Balanceo de Carga:** Nginx distribuye las solicitudes entrantes entre varios servidores backend de KAVAC, asegurando que ninguna máquina se sobrecargue y que los recursos se utilicen de manera eficiente.
- **Escalabilidad Horizontal:** Permite añadir o quitar servidores backend fácilmente sin interrumpir el servicio, lo que facilita la escalabilidad de la aplicación.
- **Terminación SSL/TLS:** Nginx maneja la terminación de SSL/TLS, asegurando que todo el tráfico entre los usuarios y el servidor esté encriptado y protegido contra ataques de intermediarios.
- **Oculto la Infraestructura:** Al actuar como intermediario, Nginx oculta la infraestructura del backend, haciendo más difícil para los atacantes dirigirse a los servidores de la aplicación.
- **Caché de Contenido Estático:** Nginx puede almacenar en caché contenido estático como imágenes, scripts y hojas de estilo, reduciendo la carga en los servidores backend y mejorando los tiempos de respuesta para los usuarios.
- **Compresión de Datos:** Nginx puede comprimir respuestas antes de enviarlas al cliente, lo que reduce el uso del ancho de banda y acelera la entrega de contenido.
- **Persistencia de Sesión:** Asegura que las solicitudes de un mismo usuario se dirijan al mismo servidor backend, lo cual es esencial para mantener la consistencia en las sesiones de usuario.
- **Manejo de Conexiones WebSocket:** Facilita la gestión de conexiones WebSocket para aplicaciones en tiempo real, manteniendo las conexiones abiertas y permitiendo la comunicación bidireccional.
- **Logs Centralizados:** Nginx registra todas las solicitudes y respuestas, proporcionando un único punto para monitorear el tráfico y diagnosticar problemas.

Lección 2: Certificados SSL Autofirmados

¿Qué es un certificado SSL autofirmado?

Es un certificado que no ha sido emitido por una autoridad de certificación (CA) reconocida, sino que ha sido firmado por la propia entidad que lo crea. Esto significa que, aunque

Licencia de Contenidos de Cenditel: <http://derechoinformatico.cenditel.gob.ve/licencia-de-contenido/>

encripta la información de la misma manera que un certificado emitido por una CA, no ofrece la misma verificación de la identidad del servidor.

Características:

- **Costo:** Son gratuitos, ya que no requieren el pago de tarifas a una autoridad de certificación.
- **Facilidad de Creación:** Se pueden generar rápidamente usando herramientas comunes como OpenSSL.
- **Control Completo:** El propietario del certificado tiene control total sobre su creación y gestión.

Recomendaciones:

Desde CENDITEL recomendamos utilizar certificados válidos que sean emitidos por una entidad de certificación que cumpla con los estándares de seguridad.

Para efectos de prueba, en la instalación del kavac hacemos uso de openssl (certificado autofirmado), sin embargo, se hace la salvedad para entornos de producción donde las conexiones deben ser seguras y la transferencia de datos deben estar encriptadas. Por lo tanto, es recomendable hacer uso de un certificado emitido por una entidad certificadora reconocida que brinda a su vez confiabilidad sobre la autenticidad del sitio web.

Utilizar certificados autofirmados cumple con la función de encriptar datos pero pueden tener limitaciones como:

Menor Nivel de Confianza: Los navegadores y clientes mostrarán advertencias de seguridad porque no pueden verificar la identidad del servidor. Los usuarios podrían desconfiar al ver advertencias de seguridad, lo que puede afectar negativamente la percepción y el uso de la aplicación.

Licencia de Contenidos de Cenditel: <http://derechoinformatico.cenditel.gob.ve/licencia-de-contenido/>

Lección 3: Gestión de Logs

Cuando no hay servicio de resolución DNS:

En caso de no tener un servicio de resolución de DNS para la publicación del kavac se puede utilizar el archivo host para darle acceso a cada uno de los equipos que necesiten acceder al kavac.

Pasos:

En cada equipo se debe abrir un terminal como super usuario.

Abrir el archivo /etc/hosts

agregar la siguiente configuración al archivo

* Nombre del dominio con el que publicamos el kavac

* ip del servidor proxy

ejemplo: kavac.gob.ve 192.168.122.88

Guardar y cerrar y acceder a través del navegador al dominio kavac.gob.ve

Nota: este procedimiento lo debes hacer en cada equipo que este dentro de la red en la que esta el servidor y del cual se desee acceder al kavac, solo para el caso que no cuente con un dominio.

Recomendaciones:

Es importante mencionar que este tipo de configuraciones se realiza solo en pruebas cuando no se tiene el dominio habilitado y se necesita visualizar en el navegador que el sistema se ha desplegado correctamente. Se recomienda que para una infraestructura de producción se cuente con un servicio de resolución de DNS.

Logs de NGINX:

Nginx, como servidor web y proxy inverso genera logs que son cruciales para monitorear el rendimiento, la seguridad y el correcto funcionamiento de las aplicaciones. Los logs de Nginx se dividen principalmente en dos tipos:

- * Logs de acceso
- * Logs de error.

A continuación, se detalla cada tipo y su importancia, así como ejemplos de configuración.

- Logs de Acceso:

Propósito: Registran cada solicitud que recibe el servidor Nginx, incluyendo información sobre el cliente que realiza la solicitud, el recurso solicitado, el estado de la respuesta, el tiempo que tardó en procesarse la solicitud, entre otros.

Ubicación Predeterminada: /var/log/nginx/access.log

- Logs de Error:

Propósito: Registran cualquier error que ocurra mientras Nginx procesa las solicitudes. Esto incluye errores de configuración, problemas de conectividad, errores de permisos, y cualquier otra falla que impida el correcto funcionamiento del servidor.

Ubicación Predeterminada: /var/log/nginx/error.log

Para el caso del kavac los configuramos con el nombre kavac-error.log y kavac-acces.log.

Logs de uwsgi:

El log de uWSGI es fundamental para monitorear el rendimiento, depurar errores y asegurar el funcionamiento eficiente de las aplicaciones web.

- Log de aplicación:

Propósito: Registra todas las solicitudes HTTP procesadas por uWSGI, incluyendo información sobre el cliente, el recurso solicitado, el estado de la respuesta y el tiempo de procesamiento.

Licencia de Contenidos de Cenditel: <http://derechoinformatico.cenditel.gob.ve/licencia-de-contenido/>

Ejemplo de Entrada: Fri Jul 05 13:34:23 2024 - spawned uWSGI worker 1 (pid: 3098, cores: 1)

[pid: 3098|app: -1|req: -1/1] 127.0.0.1 () {70 vars in 1416 bytes} [Fri Jul 05 13:34:23 2024]
GET /login => generated 10478 bytes in 1976 msecs (HTTP/1.0 200) 16 headers in 1508 bytes (1 switches on core 0)

[pid: 3098|app: -1|req: -1/2] 127.0.0.1 () {68 vars in 1756 bytes} [Fri Jul 05 13:34:23 2024]
GET /captcha/default => generated 11489 bytes in 688 msecs (HTTP/1.0 200) 17 headers in 1516 bytes (1 switches on core 0)

Ubicación Predeterminada: /var/log/uwsgi/app/nameapp.log

El registro de la bitácora se ubica en la ruta antes señalada, donde el nombre nameapp.log, lo toma automáticamente del nombre de archivo de configuración .ini de la aplicación que se ha configurado posterior al reinicio de uwsgi. Ej, nombre de archivo kavac.ini, por tanto, el nombre para el log es kavac.log.

Logs de Laravel:

Los logs de Laravel registran y rastrear eventos que ocurren dentro de la aplicación ya sean errores, advertencias, otros; que permiten identificar incidentes para la pronta resolución de problemas.

Estos registros ayudan a :

- Errores y excepciones: Cuando ocurre un error o una excepción en tu aplicación, se registra información detallada sobre el error, como la línea de código donde ocurrió y el mensaje de error.
- Eventos: Algunos eventos específicos de la aplicación, como la creación o eliminación de usuarios, backup de la base de datos por la interfaz, entre otros.
- Estos logs los encuentras dentro de la aplicación en la ruta /storage/logs/ y se registran con el nombre laravel-año-mes-día y dentro del archivo del log muestra la hora en que se generó el evento o error de la aplicación.

Licencia de Contenidos de Cenditel: <http://derechoinformatico.cenditel.gob.ve/licencia-de-contenido/>

Nota: En caso que la aplicación falle y en este directorio no se registra incidencia alguna, se deben revisar los logs de uwsgi y nginx (web y proxy).

Referencias:

nginx. (s.f.). <https://nginx.org/en/>

Licencia de Contenidos de Cenditel: <http://derechoinformatico.cenditel.gob.ve/licencia-de-contenido/>

